

✓ General

[Collapse all](#) [Avisos](#) ○ [Plano de Ensino](#) ○ [Avaliação](#) ○

Nota Final = $EP \cdot 0.3 + AT \cdot 0.3 + TP \cdot 0.4$

Onde:

EP = Exercícios Práticos (em aula)

AT = Avaliação dos Tópicos (questionários)

TP = Trabalhos Práticos (extraclasse)

✓ Tópico 1 - Introdução à Análise de Algoritmos

Apresentação

No primeiro tópico vamos conhecer a disciplina de Projeto e Análise de Algoritmos e entender como será a dinâmica do curso, além de conhecer a bibliografia, ementa e objetivos.



Objetivo

Os alunos deverão aprender sobre conceito básicos de Complexidade de Algoritmos e reconhecer a importância da disciplina para sua formação.

- Compreender como a disciplina de Projeto e Análise de Algoritmos será útil;
- Conhecer as bibliografias Básicas e Complementares;
- Aprender o que será ensinado ao longo do semestre;
- Fixar os conceitos básicos sobre Complexidade de Algoritmos



Referência para Estudo

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 4. ed. Rio de Janeiro: LTC, 2024. **Parte IV**

CORMEN, Thomas H. Desmistificando algoritmos. Rio de Janeiro: Elsevier, 2014. **Capítulo 2**

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H; VAZIRANI, Umesh Virkumar. Algoritmos. São Paulo: McGraw-Hill, 2009. **Capítulo 2**

 [Plano de Ensino e Introdução](#) ○[✓ Done](#)

[Introdução - Exercícios](#)

A) Elabore o melhor algoritmo para receber uma sequência de n números inteiros. Depois o algoritmo deve receber um número m e deve trazer como saída o número de vezes que o valor m apareceu nesta sequência.

- Considere $n < 1.000.000$
- NOTA: existe alguma consideração diferente caso m seja um inteiro entre 0 e 10.000, ou um inteiro entre 0 e 1.000.000.000.000?
- OBS: e se o valor de m fosse informado antes da sequência de n números?

B) Considere que cada operação leva 1ns em média em um determinado processador. Determine o tempo das funções $\log_2 n$, n , $3n$, $n \log_2 n$, n^2 , 2^n e $n!$ para as seguintes quantidades de operações: 10, 100, 1.000, 10.000, 100.000 e 1.000.000.

[Programação básica](#)

✓ Done

[Análise de Complexidade](#)

✓ Done

[Análise de Complexidade - Exercícios](#)

A) Implemente as funções de ordenação analisadas e faça um quadro comparativo do tempo de execução para ordenar:

- $(n=)100.000$ números aleatórios
- $(n=)100.000$ números já em ordem crescente

B) Elabore os seguintes algoritmos e analise o seu tempo de execução para diferentes entradas e determine a sua complexidade de tempo:

- Receber como parâmetro dois valores inteiros a e b e calcula a^b .
- Receber duas matrizes quadradas de mesma ordem ($n \times n$) e realiza a multiplicação entre elas.

[Avaliação Tópico 1](#)

✓ Done

Questões finais do tópico 1 (AT₁)

[Trabalho Prático 1 \(TP-1\)](#)

✓ Done

Trabalho Prático 1 – Algoritmo de ordenação simples (BubbleSort)

O objetivo deste trabalho é permitir ao aluno implementar um algoritmo de ordenação visto em aula e apresentar uma [avaliação](#) numérica do desempenho em diferentes casos de teste.

Descrição do Trabalho:

O trabalho consiste em implementar e comparar o tempo de execução (em número de operações) do algoritmo de ordenação BubbleSort visto em sala. O mesmo deve ser capaz de ordenar diversos vetores de números inteiros. A entrada do programa é o nome de um arquivo texto contendo vetores de diferentes tamanhos. O conteúdo dos arquivos de teste são valores inteiros (um valor por linha). A última linha do arquivo é uma linha em branco indicando que terminou a lista numérica. Ou seja, não é apresentado a priori a quantidade de valores. Os arquivos de teste (usado pelo professor para [avaliação](#) do trabalho) podem conter 100, 1000, 10.000 ou 100.000 valores.

Serão testados três diferentes tipos de vetores: aleatórios, ordenados em ordem crescente, e ordenados em ordem decrescente. Você deve criar arquivos neste padrão para testar sua implementação.

A saída do programa NÃO é a lista de valores ordenados. A saída esperada do programa é um valor numérico informando a quantidade de operações realizadas para ordenar o vetor.

Exemplo:

Seu programa receberá como parâmetro o nome do arquivo e retornará como saída (na tela/terminal) a quantidade de operações necessárias para realizar a ordenação. A seguir alguns exemplos de saída:

```
> bubblesort ordenado_100.txt
```

Número de operações: 99

```
> bubblesort randomico_1000.txt
```

Número de operações: 855627

```
> bubblesort decrescente_10000.txt
```

Número de operações: 49995000

 [Jupyter \[IDE online\]](#)

 [Kaggle \[IDE online\]](#)

 [Google Colab \[IDE online\]](#)

 [Visual Studio Code \[VSCode online\]](#)

 [WinLibs standalone build of GCC and MinGW-w64 for Windows](#)

 [Complexidade de diferentes algoritmos](#)



✓ Tópico 2 - Análise de Complexidade

Análise de Complexidade

No segundo tópico vamos realizar a [análise de complexidade](#) de [algoritmos recursivos](#), estudar a complexidade de espaço, a notação assintótica e teorema mestre. Também serão avaliados [algoritmos eficientes de ordenação](#) (classificação).



Objetivo

Os alunos deverão aprender sobre conceito básicos de Análise de Algoritmos e avaliar a complexidade de **algoritmos recursivos** e **algoritmos eficientes de ordenação**.

- Aprender o que é **complexidade de espaço, notação assintótica e teorema mestre**;
- Avaliar a complexidade de **algoritmos recursivos**;
- Calcular a complexidade de algoritmos de ordenação eficientes;
- Fixar os conceito de Complexidade de Algoritmos



Referência para Estudo

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 4. ed. Rio de Janeiro: LTC, 2024. [Parte II](#) e [Parte IV](#)

CORMEN, Thomas H. Desmistificando algoritmos. Rio de Janeiro: Elsevier, 2014. [Capítulo 2](#) e [Capítulo 3](#)

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H; VAZIRANI, Umesh Virkumar. Algoritmos. São Paulo: McGraw-Hill, 2009. [Capítulo 2](#)



[Algoritmos Recursivos](#) ○

✓ Done



[Algoritmos Recursivos - exercícios](#)

1. Escreva duas versões do algoritmo de fibonacci: a versão "ruim" apresentada e uma versão "boa" usando vetores. Faça uma comparação de tempo de execução com valores entre 25 e 60. Qual a complexidade da versão "boa" do algoritmo?

2. Elabore um algoritmo recursivo para calcular a raiz quadrada de um número real (positivo) qualquer. Escreva a relação de recorrência e a respectiva complexidade do algoritmo para o pior caso.



[Complexidade de Espaço, Notação Assintótica e Teorema Mestre](#) ○

✓ Done



[Teorema Mestre - Exemplos](#) ○



[Complexidade de Espaço, Notação Assintótica e Teorema Mestre - exercícios](#)

✓ Done ▾

1. Considere o algoritmo abaixo e descreva a sua relação de recorrência, sua complexidade de tempo e espaço para o melhor e pior caso:

a)

```

int pow(int base, int exp) {
    if(exp==0) return 1;
    int ret = pow(base, exp/2);
    ret = ret * ret;
    if(exp%2 == 1)
        ret = ret * base;
    return ret;
}

```

b)

```

double foo(int* v, int n, int p){
    if(n<=0)
        return 0;
    int i, soma=0;
    for(int i=0; i<n; i+=p)
        soma += v[i];
    return sqrt(soma) + foo(v, soma%n, p);
}

```

2. Use o método mestre para fornecer limites assintóticos restritos para as seguintes recorrências:

a. $T(n) = T(n/2) + 1$

b. $T(n) = 4T(n/2) + n$

c. $T(n) = 4T(n/2) + n^3$



[Algoritmos eficientes de ordenação](#) ○

✓ Done



[Classificação em Arquivo](#) ○



[Avaliação Tópico 2](#)

✓ Done ▾

Questões finais do tópico 2 (AT₂)

Trabalho Prático 2 (TP-2)

✓ Done ▾

Trabalho Prático 2 – Comparação de algoritmos de ordenação

O objetivo deste trabalho é permitir ao aluno implementar algoritmos de ordenação de diferentes complexidades visto em aula e apresentar uma [avaliação](#) numérica do desempenho em diferentes casos de teste.

Descrição do Trabalho:

O trabalho consiste em implementar e comparar o tempo de execução (em número de operações) de diferentes algoritmos de ordenação vistos em aula. Você deve escolher 3 (três) algoritmos, sendo um deles de tempo médio quadrático (BubbleSort, InsertionSort, SelectionSort), um de tempo $O(n \log n)$ (ShellSort, MergeSort, QuickSort, Heapsort) e um terceiro de tempo linear (Countingsort, Radixsort, Bucketsort).

O programa devem ser capaz de ordenar diversos vetores de números inteiros. A entrada do programa é uma lista de nomes de arquivos texto contendo vetores de diferentes tamanhos. O conteúdo dos arquivos de teste são valores inteiros (um valor por linha). A última linha do arquivo é uma linha em branco indicando que terminou a lista numérica. Ou seja, não é apresentado a priori a quantidade de valores. Os arquivos de teste (usado pelo professor para [avaliação](#) do trabalho) podem conter 100, 1000, 10.000 ou 100.000 valores. Serão testados três diferentes tipos de vetores: aleatórios, ordenados em ordem crescente, e ordenados em ordem decrescente. Você deve criar arquivos neste padrão para testar sua implementação.

A saída do programa é um arquivo HTML contendo uma tabela com a comparação dos tempos (números de operações) para cada caso de teste em cada um dos algoritmos implementados.

Criação da tabela:

Para criação do arquivo HTML, use os seguintes algoritmos:

início do arquivo e tabela:

```
escreva "<html><head><title>PAA - Trabalho 2</title></head><body><table border=1>"
```

cabeçalho:

```
escreva "<tr><th>Arquivo</th><th>BubbleSort</th><th>QuickSort</th><th> BucketSort</th></tr>"
```

corpo para cada arquivo i (filename[i]):

```
escreva "<tr><td>" + filename[i] + "</td><td>" + bubble(filename[i]) + "</td><td>" + quicksort(filename[i]) + "</td><td>" +  
bucket(filename[i]) + "</td></tr>"
```

Final do arquivo html (e tabela):

```
escreva "</table></body></html>"
```

Exemplo

Seu programa receberá como parâmetro os nomes dos arquivos e gerará como saída um arquivo HTML contendo a comparação dos tempos (números de instruções) da execução de cada um dos três algoritmos de ordenação. A seguir um exemplo:

```
> compara ordenado_100.txt randomico_1000.txt decrescente_10000.txt
```

Arquivo resultado.html:

```
<html>

<head>

<title>PAA - Trabalho 2</title>

</head>

<body>

<table border=1>

<tr>

<th>Arquivo</th>

<th>BubbleSort</th>

<th>QuickSort</th>

<th>BucketSort</th>

</tr>

<tr>

<td> ordenado_100.txt </td>

<td> 99 </td>

<td> 9965 </td>

<td> 123 </td>

</tr>

<tr>

<td> randomico_1000.txt </td>

<td> 855627 </td>

<td> 8978 </td>

<td> 134 </td>

</tr>

<tr>

<td> decrescente_10000.txt </td>

<td> 49995000 </td>

<td> 138790 </td>

<td> 1320 </td>

</tr>

</table>

</body>

</html>
```

Saída no navegador (valores meramente ilustrativos):

Arquivo	BubbleSort	QuickSort	BuckedSort
ordenado_100.txt	99	9965	123
randomico_1000.txt	855627	8978	134
decrescente_10000.txt	49995000	138790	1320

[15 sorting algorithms in 6 minutes](#)[Animações passo a passo dos algoritmos de ordenação](#)[Algoritmos de ordenação como danças em grupo](#)

✓ Tópico 3 - Estruturas de Dados

Estruturas de Dados

No terceiro tópico vamos realizar a [análise de complexidade](#) de estrutura de dados fundamentais. Também serão avaliadas estruturas de árvores binárias, árvores auto-balanceadas e grafos.



Objetivo

Os alunos deverão avaliar a complexidade de estruturas de dados em árvore e grafos.

- Avaliar a complexidade de **estruturas de dados fundamentais**: pilha, fila, listas encadeadas, string e hash;
- Calcular a complexidade de espaço e tempo em estruturas de árvores auto-balanceadas;
- Fixar os conceitos de grafos e árvores.



Referência para Estudo

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 4. ed. Rio de Janeiro: LTC, 2024. **Parte III**, **Parte V** e **Parte VI**

CORMEN, Thomas H. Desmistificando algoritmos. Rio de Janeiro: Elsevier, 2014. **Capítulo 5**

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H; VAZIRANI, Umesh Virkumar. Algoritmos. São Paulo: McGraw-Hill, 2009. **Capítulo 4** e **Capítulo 5**

[Estruturas de Dados Fundamentais](#) ○

✓ Done

[Estruturas de Dados Fundamentais - exercícios](#)

✓ Done ▾

1. Considere a sequência de chaves Q U E S T A O F C I L e a codificação A = 0, B = 1, C = 2, etc.

- Desenhe o conteúdo da tabela hash resultante da inserção dos registros com essas chaves nessa ordem em uma tabela inicialmente vazia de tamanho 7 usando listas encadeadas. Use a função hash $h(k) = k \bmod 7$ onde k é a codificação da letra.
- Desenhe o conteúdo da tabela hash resultante da inserção dos registros com essas chaves nessa ordem em uma tabela inicialmente vazia de tamanho 13 usando endereçamento aberto e hash linear para tratar colisões. Use a função hash $h(k) = k \bmod 13$ onde k é a codificação da letra.

2. Considere a implementação de uma tabela Hash de tamanho $M=11$, com endereçamento aberto utilizando a função $k \bmod M$. Responda as seguintes questões:

- Mostre a configuração da tabela após a inserção dos registros com as chaves: 4, 17, 13, 35, 25, 11, 2, 10, 32.
- Mostre a configuração da tabela após a remoção dos registros com as chaves: 25, 11.
- Mostre a configuração da tabela após a inserção dos registros com as chaves: 40, 3.

[SHA256: o que é, como funciona e para que serve? \(TOTVS\)](#)[Hash para busca de arquivos duplicados \(rmlint\)](#)

-a --algorithm=name (default: blake2b):

Choose the algorithm to use for finding duplicate files. The algorithm can be either **paranoid** (byte-by-byte file comparison) or use one of several file hash algorithms to identify duplicates. The following hash families are available (in approximate descending order of cryptographic strength):

- **sha3, blake,**
- **sha,**
- **highway, md**
- **metro, murmur, xxhash**

The weaker hash functions still offer excellent distribution properties, but are potentially more vulnerable to *malicious* crafting of duplicate files.

The full list of hash functions (in decreasing order of checksum length) is:

- 512-bit: **blake2b, blake2bp, sha3-512, sha512**
- 384-bit: **sha3-384,**
- 256-bit: **blake2s, blake2sp, sha3-256, sha256, highway256, metro256, metrocrc256**
- 160-bit: **sha1**
- 128-bit: **md5, murmur, metro, metrocrc**
- 64-bit: **highway64, xxhash.**

The use of 64-bit hash length for detecting duplicate files is not recommended, due to the probability of a random hash collision.

-p --paranoid / -P --less-paranoid (default):

Increase or decrease the paranoia of **rmLint**'s duplicate algorithm. Use **-p** if you want byte-by-byte comparison without any hashing.

- **-p** is equivalent to **--algorithm=paranoid**
- **-P** is equivalent to **--algorithm=highway256**
- **-PP** is equivalent to **--algorithm=metro256**
- **-PPP** is equivalent to **--algorithm=metro**



[Árvores Binárias e auto-balanceadas](#)

✓ Done



[Estruturas de dados em C++, Java e Python](#)

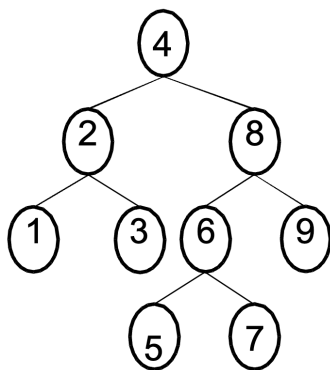


[Árvores Binárias e auto-balanceadas - exercícios](#)

Mark as done

Implemente um algoritmo que visite todos os nós de uma árvore binária com uma busca em profundidade em-ordem, pré-ordem e pós-ordem.

- Exemplo, para a árvore abaixo a saída deve ser:
 - Pré-ordem: 4, 2, 1, 3, 8, 6, 5, 7, 9
 - Em-ordem: 1, 2, 3, 4, 5, 6, 7, 8, 9
 - Pós-ordem: 1, 3, 2, 5, 7, 6, 9, 8, 4



[Árvores 2-3, B e B+](#)

✓ Done



[Árvores 2-3, B e B+ - exercícios](#)

Mark as done

1. Quantas chaves, no máximo, pode conter uma árvore 2-3 de altura 2? Qual o número mínimo de chaves em uma árvore 2-3 de altura 2? Desenhe exemplos dessas árvores.
2. Desenhe a árvore 2-3 que resulta da inserção das chaves [10, 1, 20, 30, 18, 25, 24, 11, 3], nesta ordem, em uma árvore inicialmente vazia.
3. Os dados numa árvore 2-3 são mantidos ordenados? Justifique.

4. Existe alguma relação entre a árvore Rubro-Negra e a árvore 2-3? Seria possível representar qualquer árvore Rubro-Negra em uma árvore 2-3?
5. Pesquise sobre árvores 2-3-4 (também chamada de 2-4). Verifique qual a relação entre ela e a árvore Rubro-Negra. Qual vantagem e desvantagem em relação à árvore 2-3?
6. Mostre todas as árvores B válidas com grau mínimo 2 ($M=4$) que incluem as chaves 1, 2, 3, 4 e 5.
7. Mostre o resultado da inserção (nesta ordem) das chaves 16, 29, 27, 21, 13, 22, 18, 30, 32, 33, 23, 28, 24, 26, 11, 12, 34, 35, 14, 36, 15 em uma árvore B de grau 3 ($M=6$). Mostre a configuração da árvore após cada inserção e imediatamente antes de cada split.
8. Quantas leituras são feitas no pior caso para buscar um elemento em uma árvore B. Quantas leituras e escritas são necessárias para inserir um elemento, no pior caso?



Grafos

Not available unless: The activity [Árvores 2-3, B e B+ - exercícios](#) is marked complete

Processamento de Imagens em Grafos

Not available unless: The activity [Árvores 2-3, B e B+ - exercícios](#) is marked complete

Floresta de Caminhos Ótimos

Not available unless: The activity [Árvores 2-3, B e B+ - exercícios](#) is marked complete

Imagens e Grafos

Abaixo seguem alguns links com materiais disponíveis em disciplinas sobre processamento de imagens usando grafos:

- [Página de disciplina na USP - Processamento de Imagens usando Grafos \(MAC6903\)](#)
- [Página de disciplina na UNICAMP - Processamento de Imagens usando Grafos \(MO815/MC919\)](#)
- [Apresentação sobre IFT do prof. Alexandre Falcão](#)
- Vídeo Aula sobre OPF do prof. João Paulo Papa:

Not available unless: The activity [Árvores 2-3, B e B+ - exercícios](#) is marked complete[Avaliação Tópico 3](#)

✓ Done

Questões finais do tópico 3 (AT₃)**IMPORTANTE:** Limite de tempo de **100 minutos** (1h40m)

Trabalho Prático 3 (TP-3)

Trabalho Prático 3 – Autocompletar com Árvore Trie

O objetivo deste trabalho é permitir ao aluno implementar um algoritmo para previsão de palavras a partir do início da escrita para auxílio na tarefa de autocompletar.

Descrição do Trabalho

O trabalho consiste em implementar e um algoritmo para auxílio na tarefa de autocompletar usando a estrutura de dados chamado Árvore Trie. O programa deve ler um dicionário e montar a árvore. Após lido o dicionário, o programa deve permitir que o usuário escreva o início de uma palavra. A partir do início, o programa deve sugerir quais palavras o usuário pretende escrever.

Você pode utilizar como dicionário um conjunto de palavras reservadas de uma linguagem de programação. Desta forma, o programa pode se comportar como uma IDE que auxilia o programador a digitar mais rápido. Também é possível usar como dicionário os nomes de cidade, nomes de medicamentos ou outros para ser utilizado em um programa que busca informações em um campo texto (ou lista suspensa).

A saída do programa é uma lista (em tela) de palavras do dicionário que iniciam pelas letras digitadas pelo usuário.

Exemplo

Seu programa deve ler letra a letra e exibir as possíveis palavras que o usuário queira digitar. A seguir um exemplo:

```
> minha_frutaria

Escolha uma fruta ou legume: ab<enter>

Sugestões:

abacaxi
abacate
abóbora
abobrinha
```

Not available unless: The activity [Árvores Binárias e auto-balanceadas - exercícios](#) is marked

Show more ✓



Trabalho Prático 3 (TP-3 alternativo)

Trabalho Prático 3 – Possíveis soluções para jogo **Termo**

O objetivo deste trabalho é permitir ao aluno implementar um algoritmo para previsão de palavras para auxiliar na resolução do jogo [Termo](#).

Descrição do Trabalho

O trabalho consiste em implementar e um algoritmo para auxílio na solução do jogo Termo usando a estrutura de dados chamado Árvore Trie. O programa deve ler um dicionário e montar a árvore. Após lido o dicionário, o programa deve permitir que o usuário escreva uma palavra de teste e as cores retornadas pelo website (**A**marelo, **V**erde ou **P**reto). A partir da palavra de teste (chute) e da lista de cores, o programa deve sugerir as possíveis palavras usando a árvore Trie gerada a partir do dicionário.

Adicionalmente, você também poderá utilizar as respostas anteriores para melhorar a resposta do programa. Também é possível gerar uma versão do Termo offline para fazer o computador "jogar sozinho" mostrando as tentativas e possíveis soluções de forma iterativa.

Exemplo

Seu programa deve uma palavra e um código de cores para tentar descobrir a "palavra chave":

> termo_solver

Chute #1: TERMO
resposta: APPPP

Sugestões de Palavras
(total de 182)

FALTA
PAUTA
CANTA
DATAS
LATAS
JANTA

Chute #2: PAUTA
resposta: VVUVV

Sugestões de Palavras
(total de 1)
PASTA

Chute #3: PASTA
resposta: VVVVV (resposta correta)



Not available unless: The activity [Árvores Binárias e auto-balanceadas - exercícios](#) is marked ...

Show more ▾

✓ Topico 4 - Abordagens para Resolução de Problemas

Abordagens para Resolução de Problemas



No último tópico vamos abordar os principais métodos de resolução de problemas. Também será feito o [estudo da tratabilidade de problemas computacionais](#) e redução de problemas.



Objetivo

Os alunos deverão classificar os algoritmos nos principais métodos de resolução de problemas e avaliar a possibilidade de criação de soluções mais eficientes.

- Classificar os principais métodos de resolução de problemas em relação a abordagem utilizada: Indução Matemática, Divisão e Conquista, Algoritmos Gulosos, Algoritmos de Tentativa e Erro, Programação dinâmica, Algoritmos de aproximação;
- Avaliar classes de problemas P e NP;
- Estudo de redução de problemas: SAT, Clique, Cobertura de Vértices, Ciclo Hamiltoniano, Caixeiro viajante.



Referência para Estudo

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 4. ed. Rio de Janeiro: LTC, 2024. **Parte IV**

CORMEN, Thomas H. Desmistificando algoritmos. Rio de Janeiro: Elsevier, 2014. **Capítulo 10**

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H; VAZIRANI, Umesh Virkumar. Algoritmos. São Paulo: McGraw-Hill, 2009. **Capítulo 2 e Capítulos 5 a 9**



[Criptografia RSA](#) ○

✓ Done



[RSA explained step by step \(cryptool.org\)](https://cryptool.org/)

[Criptografia RSA - exercícios](#)

✓ Done ▾

1. Sejam dois primos $p=17$ e $q=41$ usados em um esquema RSA. Qual dos parâmetros $e_1=32$ ou $e_2=49$ é um expoente válido? Calcule a chave privada correspondente.
2. Faça a criptografia e descryptografia usando o algoritmo RSA (podem usar cryptool.org):
 - $p=3$, $q=11$, $e=7$, mensagem=5
 - $p=5$, $q=11$, $e=3$, mensagem=9

[Abordagens para Resolução de Problemas](#) ○

✓ Done

[Árvore de Huffman](#) ○[Abordagens para Resolução de Problemas - exercícios](#)

✓ Done ▾

1) O pequeno Turing exagerou na quantidade de disciplinas neste semestre. Para passar nas provas finais ele precisa estudar uma quantidade de tempo para cada disciplina. Se ele estudar o tempo necessário com certeza será aprovado, caso contrário, certamente reprovará. Cada disciplina tem a sua importância subjetiva (is) para o menino, sendo que ele quer maximizar este índice de importância uma vez que sabe que não tem tempo viável para estudar para todas as disciplinas. Dada a lista de disciplinas e sua importância subjetiva, descubra qual o melhor cronograma de estudo para Turing a fim de maximizar a importância subjetiva (is) na aprovação (mostre a tabela utilizada na solução).

Tempo disponível: 16 horas

Disciplinas:

- ALP (3h – 5is); - PAP (5h – 3is);
- CAL (6h – 6is); - BAN (4h – 4is);
- REC (9h – 5is); - PPR (3h – 2is);
- TEC (8h – 6is); - SOP (4h – 4is);

2) Considere as strings *genoma* e *feature* que representam, respectivamente, uma parte do genoma de uma criatura e uma determinada característica de interesse. Deseja-se descobrir qual é a maior subsequência comum (sequência de caracteres iguais, mas não necessariamente contíguas) entre essas duas cadeias de caracteres:

genoma: ACCUGTATAUCGUCACTU

feature: GCAUTTC

[Estudo da Tratabilidade de Problemas Computacionais](#) ○

✓ Done

[Quer ganhar um prêmio e ficar milionário?](#)[Estudo da Tratabilidade de Problemas Computacionais - Exercício](#)

✓ Done ▾

[Reduções de Problemas](#) ○

✓ Done

[Avaliação Tópico 4](#)

✓ Done ▾

Questões finais do tópico 4 (AT₄)

IMPORTANTE: Limite de tempo de **100 minutos** (1h40m)

[Trabalho Prático 4 \(TP-4\)](#)

✓ Done ▾

Trabalho Prático 4 – Otimização

O objetivo deste trabalho é proporcionar ao aluno a oportunidade de selecionar um problema NP-Completo de seu interesse, com o intuito de realizar uma pesquisa e desenvolver uma solução otimizada para o problema escolhido, fundamentada em uma estratégia de redução da complexidade algorítmica.

Descrição do Trabalho

O trabalho consiste em implementar uma solução otimizada para um problema NP-Completo à sua escolha. Você deverá ser capaz de:

1. Explicar o problema escolhido.
2. Apresentar a solução original e a versão otimizada do algoritmo.
3. Detalhar as estratégias de otimização utilizadas.
4. Comparar as complexidades dos dois algoritmos (original e otimizado) usando a notação Big O.

Neste projeto, você deverá fazer **um texto em formato de artigo** (IEEE, SBC,...) e apresentar tanto os códigos gerados quanto as explicações e as análises comparativas de complexidade dos dois algoritmos. Ao final, apresente uma análise crítica das otimizações implementadas quanto à efetividade (como as otimizações afetaram o tempo de execução e a complexidade do algoritmo), limitações (se existe casos onde a otimização perde eficiência) e recomendações (quais cenários a solução otimizada é mais vantajosa e quais os *trade-offs* entre tempo de execução e precisão).

Dicas para escolha do problema:

Você pode escolher qualquer problema NP-Completo, que esteja bem documentado na literatura de algoritmos e teoria da complexidade. Algumas sugestões de problemas incluem (mas não se limitando a):

- Problema da Mochila
- Problema do Caixeiro Viajante
- Problema de Coloração de Grafos
- Problema de Satisfação Booleana (SAT)
- Problema do Ciclo Hamiltoniano
- Problema de Roteamento de Veículos
- Problema de Clique Máximo
- Problema de Conjuntos Independente



Avaliação



Avaliação da Disciplina



No campo discursivo, faça comentários para melhorar a disciplina para os próximos semestres. Indique quais tópicos não foram bem abordados, quais tópicos carecem de melhor explicação, o que acha que ficou de fora, etc. Além dos conteúdos, comente também sobre trabalhos, métodos de [avaliação](#), forma de exposição e outros.